PASS Data Community Summit

Host redgate

No Fluff, Just Facts: AI Essentials for Data Professionals



Deborah Melkin

she\her

Sr. Database Reliability Engineer

GLG

Learning Pathways – Adapting for AI Readiness as a Data Professional

- No Fluff, Just Facts: AI Essentials for Data Professionals
- AI in the SQL World: Tools We Have & Tips for Using Them
- Keeping Up Without Falling Down: AI's Future for Data Professionals



Deborah Melkin

she\her



Sr. Database Reliability Engineer

GLG

dgmelkin@gmail.com

github.com/DebtheDBA

DebtheDBA.wordpress.com

- 25 years as a DBA
- Data Platform Women in Tech (WIT) Virtual UG, Co-leader
- WITspiration, Co-founder
- Redgate Community Ambassador
- Microsoft MVP, Data Platform
- In my spare time, I can usually be found doing something musical or something geeky with my husband, Andy, and our dog, Sebastian.

Disclaimer



This slide deck was made with the help of AI.

It is the theme of this presentation, after all…

Why This Topic?



Why AI Essentials are Necessary

- Speed of change
- Need to be able to use AI effectively
  - Understand the terminology
  - Understand the different components
- AI integration is no longer a choice
  - Don’t be left behind

- In the end, AI is another ***tool*** in our skillset.

Agenda

- Key Concepts
- Ways to use AI as our Tool
- Guardrails & Security
- Walk through examples

Key Terms



Key Terms: Foundations

Deterministic

- Same input always produces the same output — unlike LLMs, which are probabilistic and can vary run to run

Token

- The basic unit of text an LLM reads and writes, roughly ¾ of a word; context limits and costs are measured in tokens

Vector

- An array of numbers representing the meaning of text or data, enabling semantic search by similarity instead of exact keyword match

Key Terms: Working with AI

Prompt

- The instruction or question you give an AI model; prompt engineering means writing prompts that reliably get the result you want

Agent

- An AI system that plans multi-step work, decides which tools to use, and acts toward a goal rather than just answering one question

Tools

- Functions an AI model can call to take action or get information beyond its own knowledge — e.g., run a query, read a file, search the web

Key Terms: Automation

Skill

- A packaged, reusable set of instructions or expertise an AI can load for a specific task, rather than reasoning from scratch every time

MCP Server (Model Context Protocol)

- A standard way to connect an AI model to external systems like databases, files, or APIs, so it can read data or take actions through one common protocol

Hook

- A checkpoint that runs custom code automatically at a defined point in an AI workflow, often used to add guardrails, logging, or validation

Model Matrix

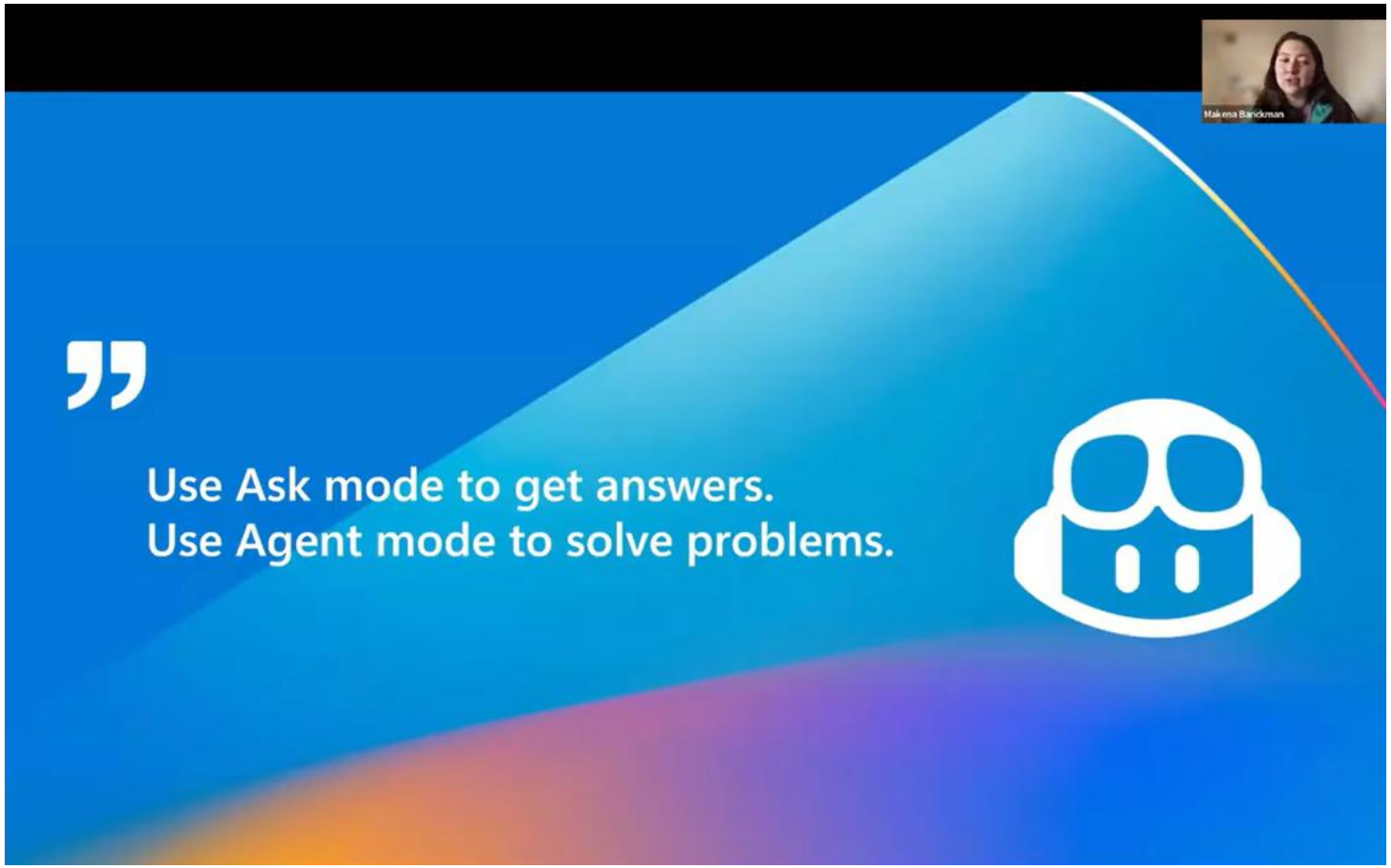
	Claude (Anthropic)	Codex (OpenAI)	GitHub Copilot
Most capable	**Opus 5.5** Complex agentic coding and research	**GPT-6.1 Sol** Complex coding and agentic workflows	**GPT-6 Astra** Long-horizon autonomous coding; also hosts Fable 5.1
Balanced	**Sonnet 5.5** Everyday coding and document work	No separate mid-tier Sol or Luna, chosen by task	**GPT-5.6 Terra** General-purpose coding and agent tasks
Fast and low cost	**Haiku 5.5** High-volume, low-latency tasks	**GPT-6 Luna** Summarization, extraction, focused coding	**GPT-6 Luna** Fast, cost-efficient help on simple tasks

Lineups as of October 2026. Copilot hosts models from several vendors, tiered by GitHub's task categories. Sources: Anthropic, OpenAI, GitHub docs.

Other AI Tools of Note

- Cursor
- Gemini
- CoCo (Snowflake's built in AI)
- < Name a tool you use and it probably has something built in or you can add one>

Chat vs Agent Modes



Makena Barickman, PM @ Microsoft

Data Platform WIT UG, Sept 2026

Translating Current Workflow into AI



Start With a Standard Runbook

Failover Runbook

- [x] Verify backups completed
- [x] Check replication lag
- [x] Run pre-failover validation
- [ ] Execute failover
- [ ] Confirm app connectivity

Index Maintenance Runbook

- [x] Identify fragmented indexes
- [x] Check maintenance window
- [ ] Rebuild / reorganize indexes
- [ ] Update statistics
- [ ] Verify job completion

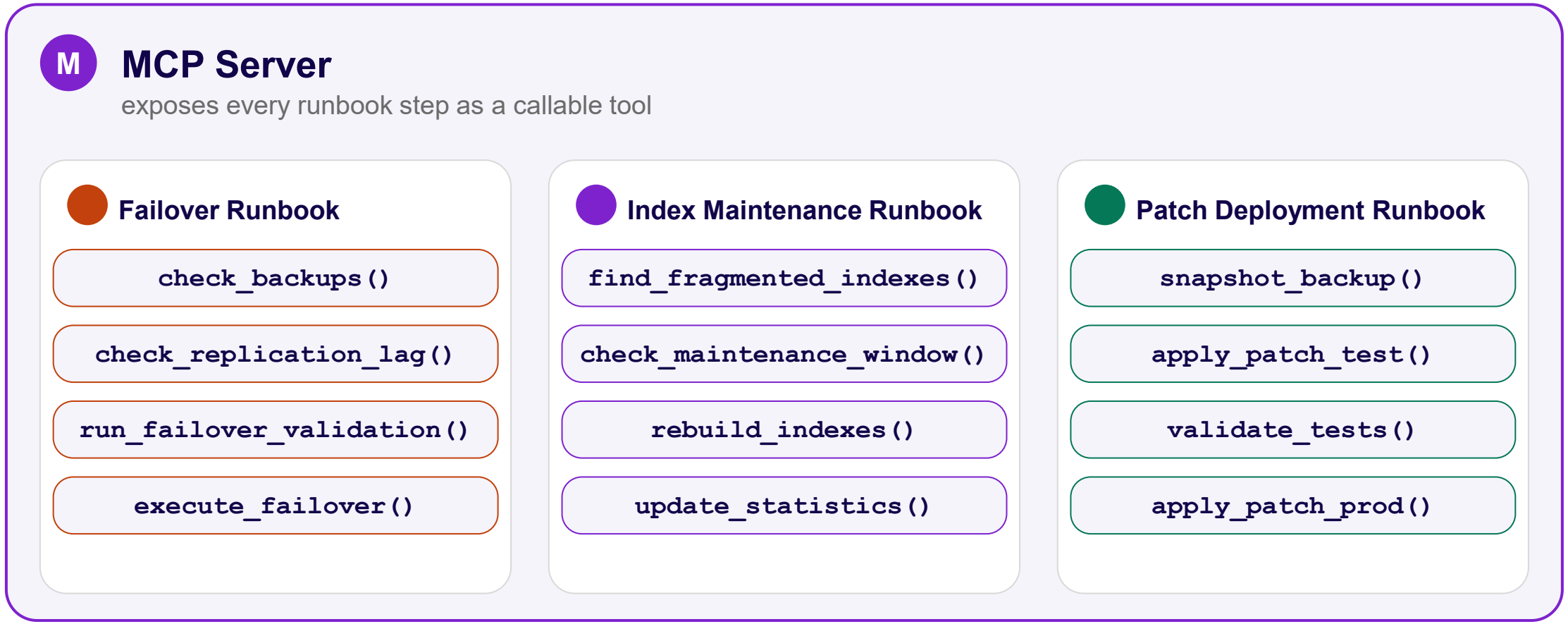
Patch Deployment Runbook

- [x] Snapshot / backup instance
- [x] Apply patch to test instance
- [ ] Validate application tests
- [ ] Apply patch to production
- [ ] Confirm rollback plan ready

Same checklist, different implementation:

`.sql scripts` `Jupyter notebook` `PowerShell (calling PowerShell scripts)` `Python script`

From Runbooks to an MCP Server



Each checkbox becomes a tool the MCP server exposes — every runbook becomes a related set of tools.

A Skill Points to an MCP Server

SKILL.md

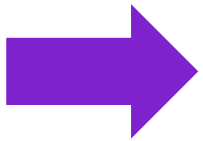
```
---
name: failover-runbook
description: Run the failover
  runbook via the Failover MCP server
---

# Failover Runbook

Use the Failover MCP Server tools
to complete each step, in order:

1. check_backups
2. check_replication_lag
3. run_failover_validation
4. execute_failover
```

points to



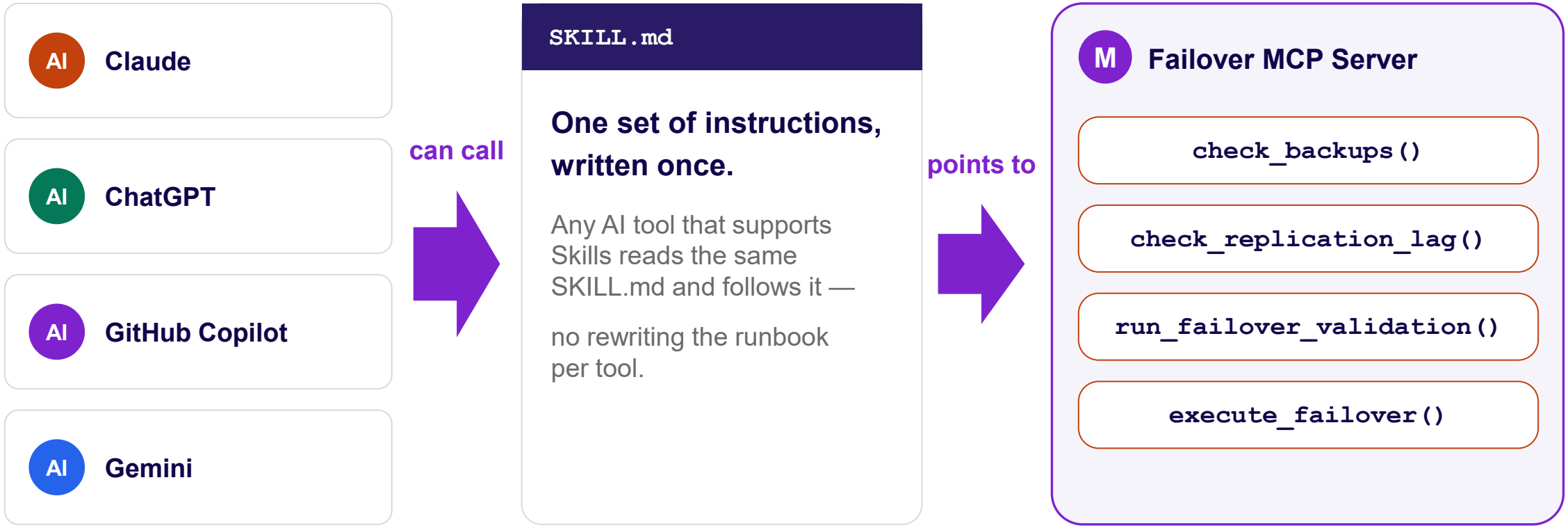
M Failover MCP Server

- `check_backups()`
- `check_replication_lag()`
- `run_failover_validation()`
- `execute_failover()`

= the Failover Runbook, exposed as tools

A Skill packages the instructions; the MCP server it points to supplies the tools that actually do the work.

Any AI Tool* Can Call the Skill



The Skill is the portable layer — swap the AI tool *, keep the same runbook and the same MCP server.

Why the * on the previous slide

- Each AI tool will have different directories and names to store skills, settings, etc.

	Claude Code	OpenAI Codex	GitHub Copilot
Instructions	`CLAUDE.md` `~/.claude/CLAUDE.md`	`AGENTS.md` `~/.codex/AGENTS.md`	`.github/copilot-instructions.md` `AGENTS.md also read`
Skills	`.claude/skills/*/SKILL.md` `~/.claude/skills/`	`.agents/skills/*/SKILL.md` `~/.agents/skills/`	`.github/skills/*/SKILL.md` `~/.copilot/skills/`
Custom agents	`.claude/agents/*.md` `~/.claude/agents/`	`.codex/agents/*.toml` `~/.codex/agents/`	`.github/agents/*.agent.md` `~/.copilot/agents/`
Settings	`.claude/settings.json` `~/.claude/settings.json`	`.codex/config.toml` `~/.codex/config.toml`	`.vscode/settings.json` `~/.copilot/config.json`
MCP servers	`.mcp.json` `~/.claude.json`	`[mcp_servers] in config.toml` `~/.codex/config.toml`	`.vscode/mcp.json` `~/.copilot/mcp-config.json`

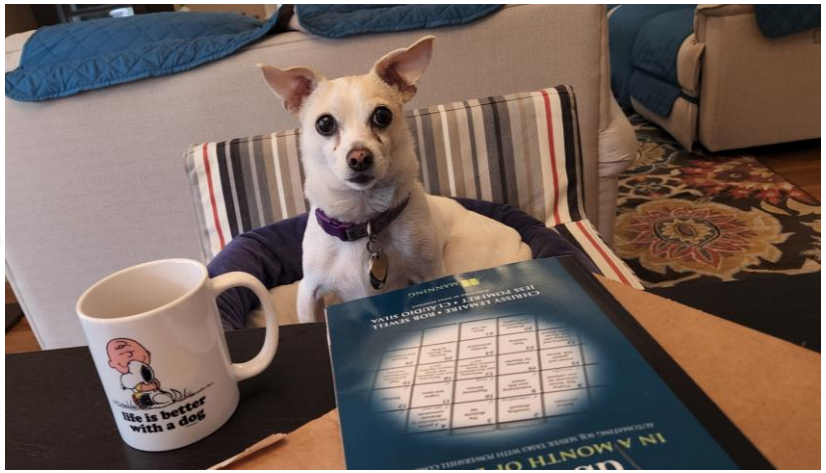
Top line: project path. Gray line: user-level path in your home folder. Copilot also reads .claude/skills and .agents/skills.

Ways that we use AI as our Tool



Sounding Board / “Rubber Duck”

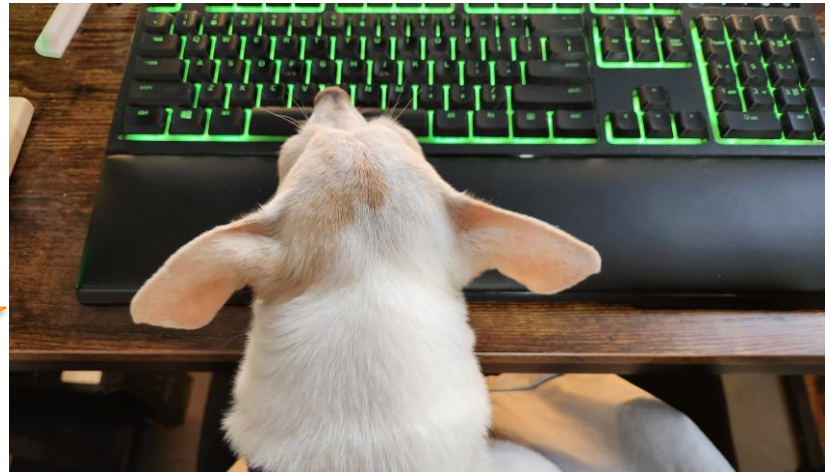
Tutor for something unfamiliar





Peer Programmer

Documentation / Junior Dev / Intern



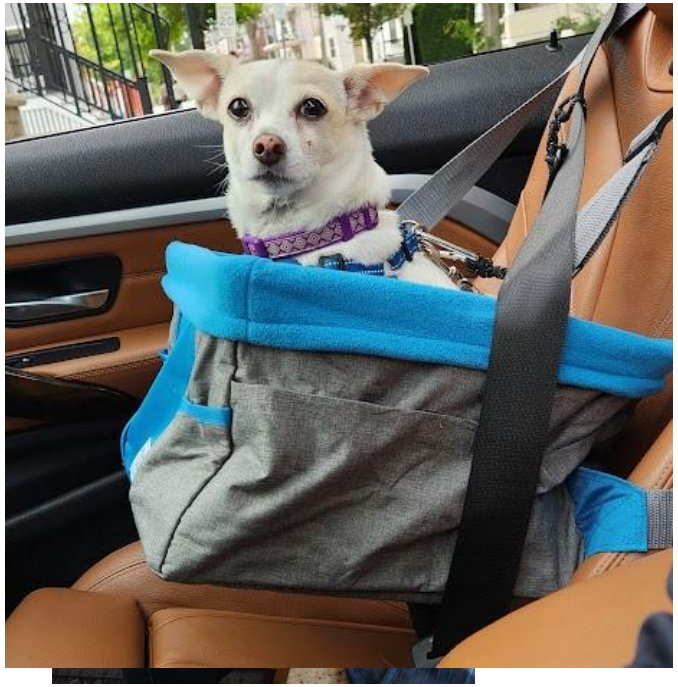
Guardrails

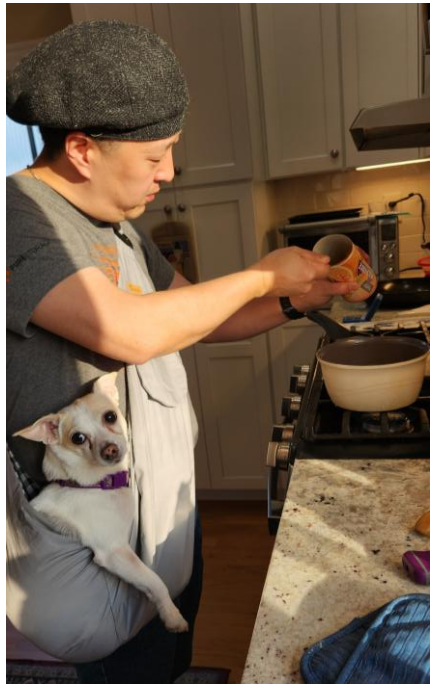


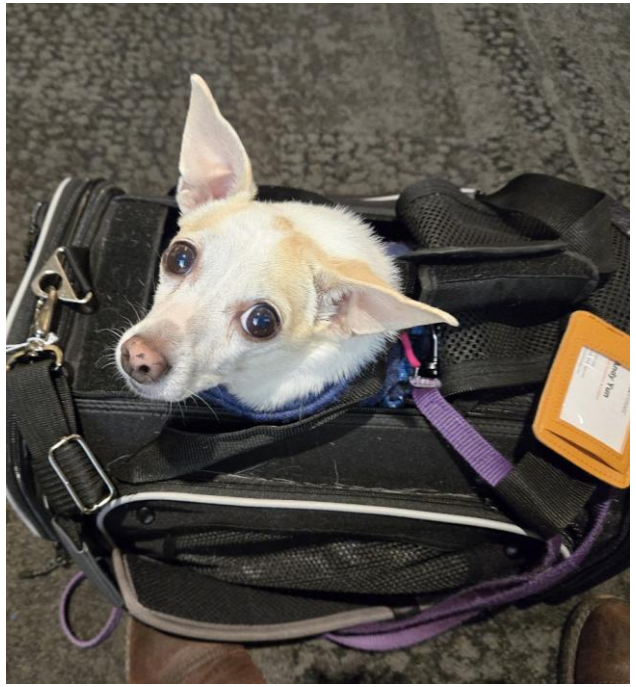


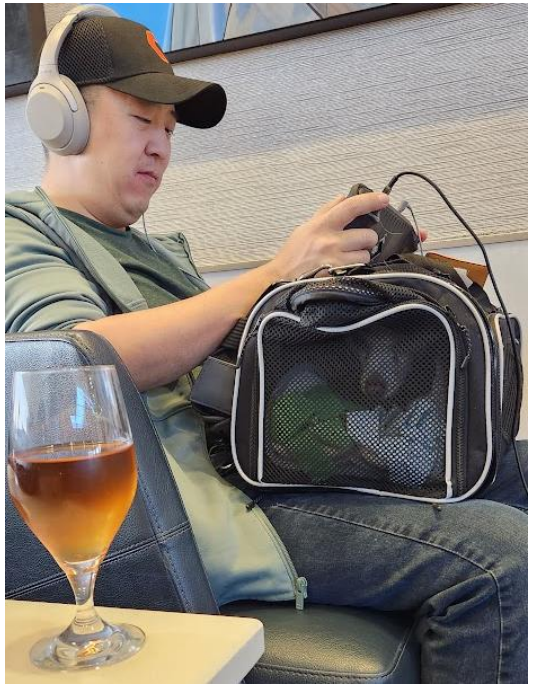



What we really want to accomplish:









Security is not with AI but the infrastructure?

- Security Flaws exist
- AI does a good job at exposing & exploiting them

- Do you know who your AI is connecting to your database as?

Tips for Securing Your AI

- Designate a login and set permissions
- Set the App Name for tracing
- Use MCP servers to help control actions
- Review the AI's code
  - Have AI review the AI's code for security leaks
- Limit what it can do on its own
  - Hooks
  - Always ask permission

Now it's time to look at some of these things we've talked about...



Let's use AI as a Sounding Board/"Rubber Ducking"

- What Does AI Have to Say For Itself?

- We have a topic that we're researching so let's see what AI's thoughts are...







The Prompt:



- If you were a database professional, what are the most important things that you think you should know about incorporating AI into daily work?
- Please return the results as a PowerPoint slide or a slide that can be added to a PowerPoint deck. Multiple slides may be acceptable.
- In the footer, add a comment saying what AI tool generated the response.

ChatGPT Response



Where AI helps a database professional

Use it as a draft and diagnostic partner for work you can verify.

01	Query development	Draft T-SQL, dbt models, test cases, or migration scripts from a precise specification.
02	Performance investigation	Summarize an execution plan or workload pattern, then propose hypotheses to measure.
03	Operations and communication	Turn runbook steps, incident notes, and schema changes into clear drafts for review.

Sources
Microsoft Learn: Accelerate Azure SQL tasks with Microsoft Copilot
Microsoft Learn: Transparency Note for Copilot in SSMS

The verification loop

AI output is a candidate. Database behavior is the evidence.

01	Give usable context	State engine and version, schema, row volumes, indexes, expected result, and constraints.
02	Inspect the proposal	Check joins, NULL handling, permissions, transaction scope, and destructive effects.
03	Measure before release	Run tests on representative data, compare plans and resource use, then review deployment and rollback.

Sources

Microsoft Learn: Accelerate Azure SQL tasks with Microsoft Copilot
Microsoft Learn: Transparency Note for Copilot in SSMS
NIST: AI Risk Management Framework Core

Claude Response



Eight things every database professional should know

The AI writes the draft. You own the result.

Treat output as untrusted

Review every query, proc, and script like a junior's pull request. "It runs" is not "it's right."

Context is the real skill

Give it schema, keys, naming standards, and business rules, and store them where the tool reads them.

Protect the data

Know where prompts go and what's retained. Keep PII and prod extracts out of unapproved tools; mask or synthesize.

Least privilege for AI

AI connections and agents get read-only, scoped logins. DDL/DML goes through a human and change control.

Correct ≠ efficient

Generated SQL often returns the right rows with a bad plan. Check the plan, estimates, and cost before it ships.

Curate what AI can query

Natural-language-to-SQL is far more reliable over curated views or a semantic layer than over raw tables.

Keep fundamentals sharp

You only catch mistakes you understand. Indexing, isolation levels, and plan reading matter more now, not less.

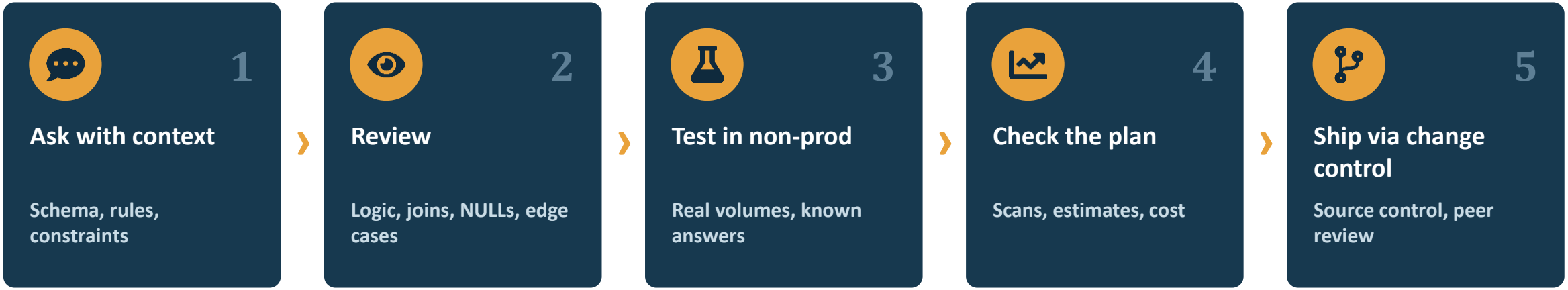
Leave an audit trail

Commit AI-assisted changes to source control and note AI involvement, so reviews and incidents stay traceable.

Generated by Claude Opus 5.5 (Anthropic) · September 2026

A safe daily loop

Same five steps whether it's a one-line query or an agent-generated migration.



New ground worth learning now (SQL Server examples, as of Sept 2026)

- GitHub Copilot in SSMS 22 (GA Nov 2025)
- Copilot Agent Mode (SSMS 22.7 preview, Jun 2026)
- Native VECTOR type + DiskANN indexes (SQL Server 2025)
- SQL MCP Server for AI agents (preview)

Gemini Response

NOTE: This was not able to generate slides for me, even Google Slides.

Core Principles, Opportunities, and Guardrails for SQL Server & PostgreSQL Environments

Assisted Development vs. Execution Reality

- Use LLMs for T-SQL/PL/pgSQL drafting, synthetic data generation, and regex development.
- Always validate generated queries against execution plans (SQL Server Graphical/XML Showplan, PostgreSQL EXPLAIN (ANALYZE, BUFFERS)).

Vector Search & Embedding Operations

- Modern workloads require managing high-dimensional embeddings directly in relational engines.
- Contrast implementations: native VECTOR types and DiskANN in SQL Server 2025 vs. the open-source pgvector extension (HNSW / IVFFlat indexing) in PostgreSQL.

Autonomous Tuning & Query Drift Observability

- Pair machine learning anomaly detection with built-in database telemetry.
- Correlate engine-native observability tooling: SQL Server Query Store & Automatic Tuning vs. PostgreSQL pg_stat_statements & autonomous tuning daemons.

Data Governance, Boundary Isolation & Security

- Prevent leakage of proprietary schemas, stored procedure logic, and PII to public model providers.
- Enforce role-based access control (RBAC), Dynamic Data Masking (DDM), and enterprise data posture frameworks (e.g., Microsoft Purview).

Generated by Gemini 3.8 Flash

What do you think of the responses?



Additional Observations

- None of these were created with a single prompt. There were multiple steps in the conversation.
- The questions asked by each AI were different.

What is this telling us?

Different tools

\+ Different models

\+ Different capabilities

= Different results

Test the various models to find what works best

What are some of the things you're struggling with?



Let’s Jump into the Tools



Final Thoughts



Final Thoughts

- We've had to learn how to "Google" or "Bing". Now we have to learn how to "Prompt"
- AI is a tool. We need to learn when it's appropriate
- Create instructions and personas
- Spend time learning how it integrates with your existing tools and workflows
- Like anything new, start small and then grow from there
- Don't trust and always verify
- Things may not always be consistent

- This slide deck is out of date already. It's changing that fast...

Resources



Resources

Outside Sessions

- What's New in SSMS 22, Makena Barickman
- Building a DBA Agent for Your SQL Server Estate, Anthony Nocentino
- Enterprise Vibe Coding, Chrissy LeMaire

Documentation and frameworks

- Accelerate Azure SQL tasks with Microsoft Copilot, Microsoft Learn
- Transparency Note for Copilot in SSMS, Microsoft Learn
- GitHub Copilot in SSMS Best Practices, Microsoft Learn
- AI Risk Management Framework Core, NIST
- Top 10 for LLM and Gen AI Applications (2025), OWASP

The slidedeck for this presentation:

- Github repo for this session: DebtheDBA/AIEssentialsForDataProfessionals

Resources

AI model references

- Anthropic Claude Timeline, ScriptByAI
- Claude Models, Gradually.ai
- Claude Release Notes, Anthropic
- Codex Models, OpenAI
- Introducing GPT-5.5, OpenAI
- Supported AI Models in GitHub Copilot, GitHub Docs
- AI Model Comparison, GitHub Docs

Thank you

Any Final Questions?



Deborah Melkin

dgmelkin@gmail.com

github.com/DebtheDBA

DebtheDBA.wordpress.com